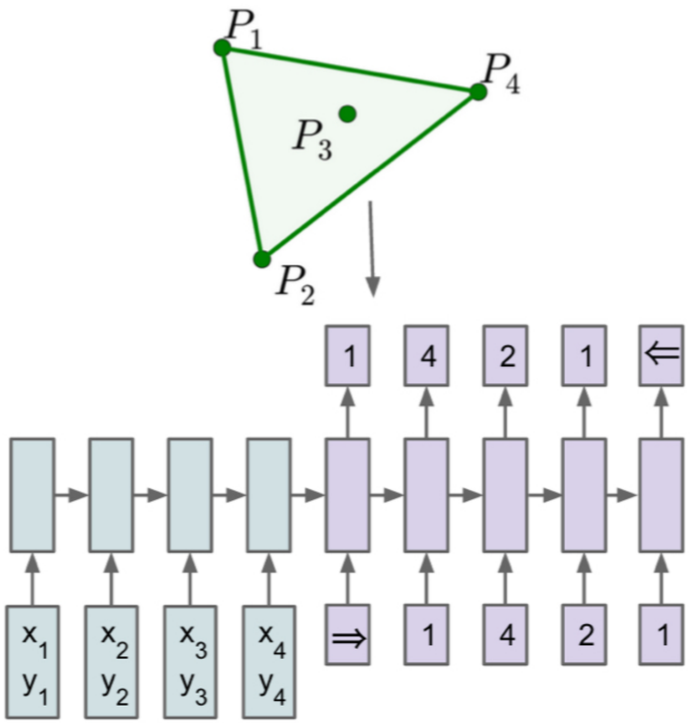
The road to Transformer Emergence

Pointer Network -> Set2Set -> Relational Network

수리과학부25 윤승현 + Chatgpt

Table of contents

Pointer Network



-입력으로 준 토큰을 pointing하는 문제들 있음.

- Convex Hull, Delaunay Triangulation, TSP

- NTM에 등장한 variable-length 문제를 끌고와서 생각해보자.

- 토큰-숫자 일대일 대응이 아니라면 학습의 어려움이 있을수도 -> 숫자에 대한 학습도 이루어져야 함.

Cognitive Science and Linguistics

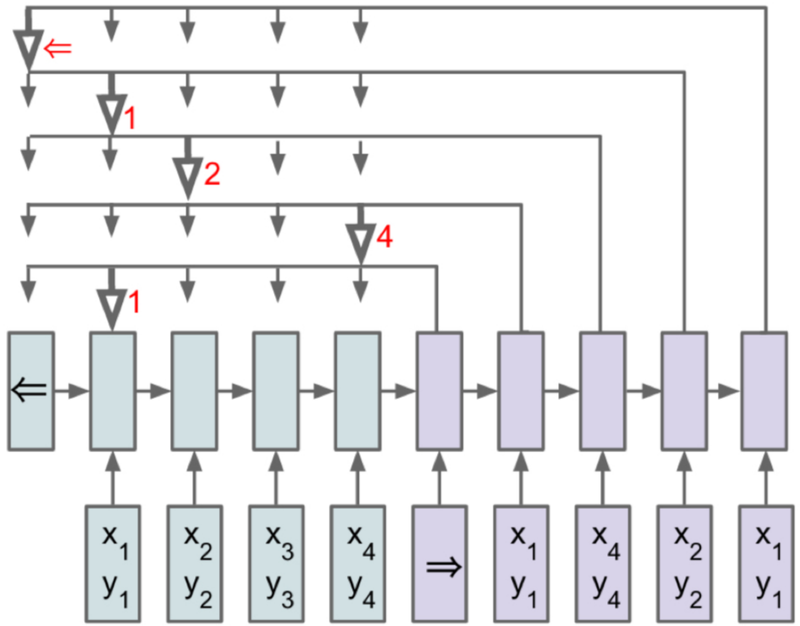
Fodor and Pylyshyn’s limitation of neural networks for cognitive modeling(1988)

Variable-binding and Variable-length

fixed length input domain으로 정적인 length

(recursive processing of variable-length structure이 인류 인지의 상징으로 여겨짐)

Pointer Network



(b) Ptr-Net

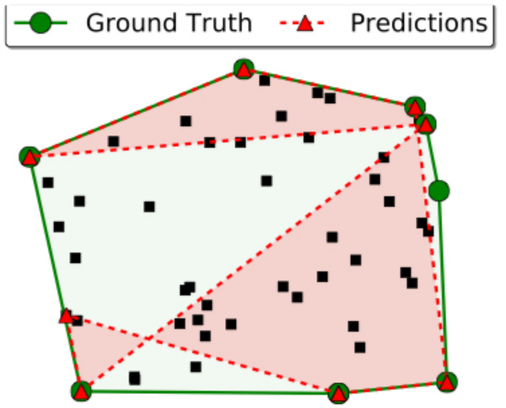
정말 기발한 아이디어!!!!

- ‘RNN search와 유사하게’ **입력**을 pointing!

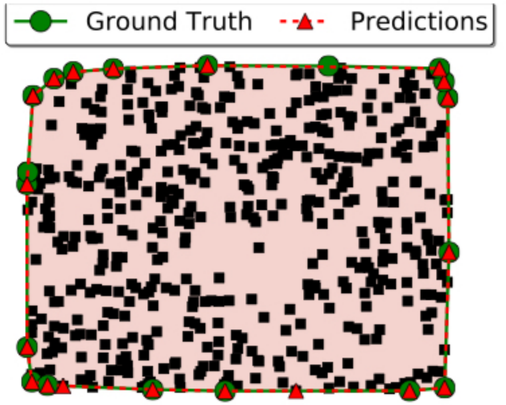
Pointer Network

-convex hull에서 factor 10의 일반화!

-다른 문제에서는 조금 더 약한 일반화 관찰

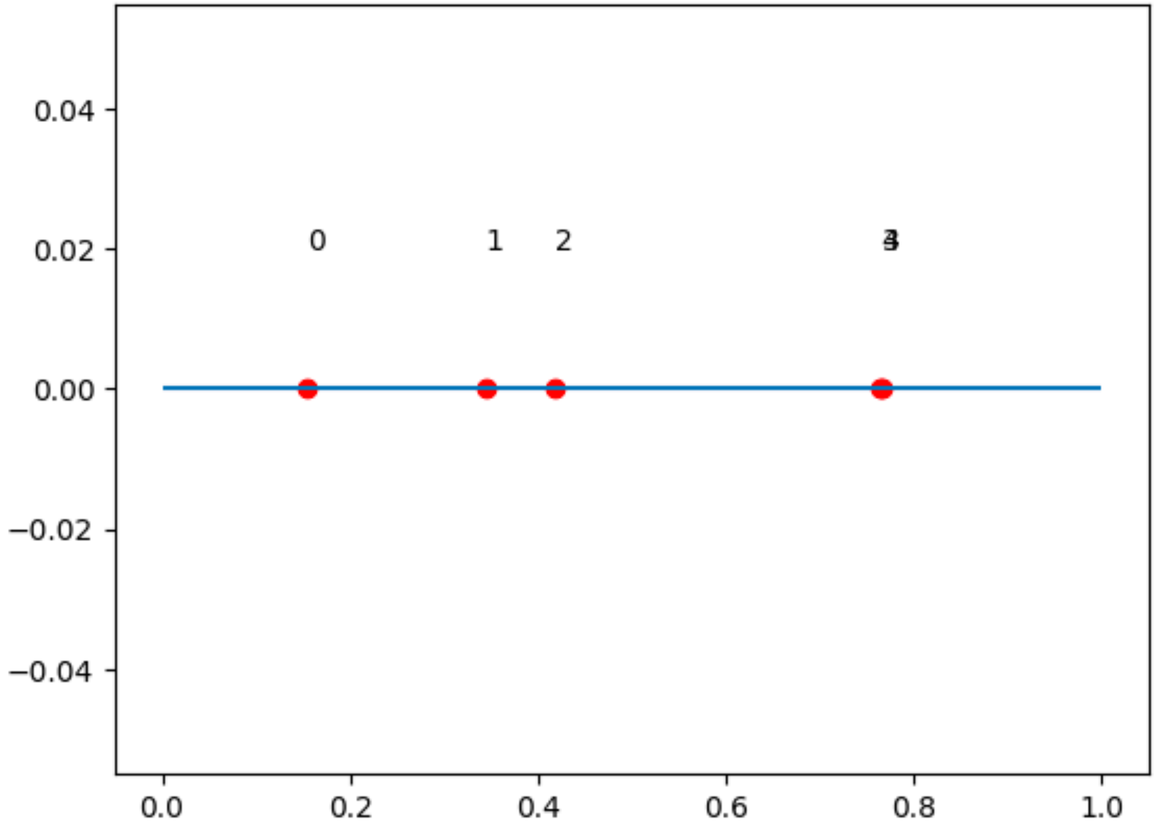


(a) LSTM, m=50, n=50



(d) Ptr-Net, m=5-50, n=500

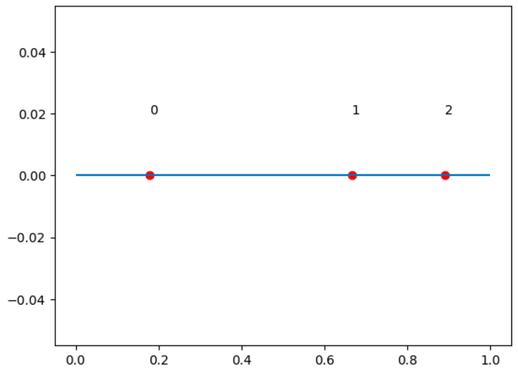
Q? on Pointer Network

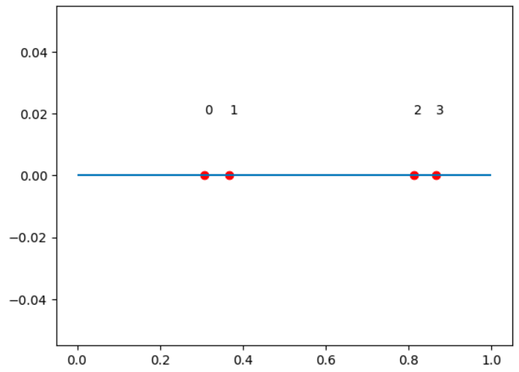


pointer network 구현해봄

- N=5인 sort 문제 / pointer 아이디어로
- 거리가 가까운 pair가 아니라면 잘 작동 👍

Q? on Pointer Network

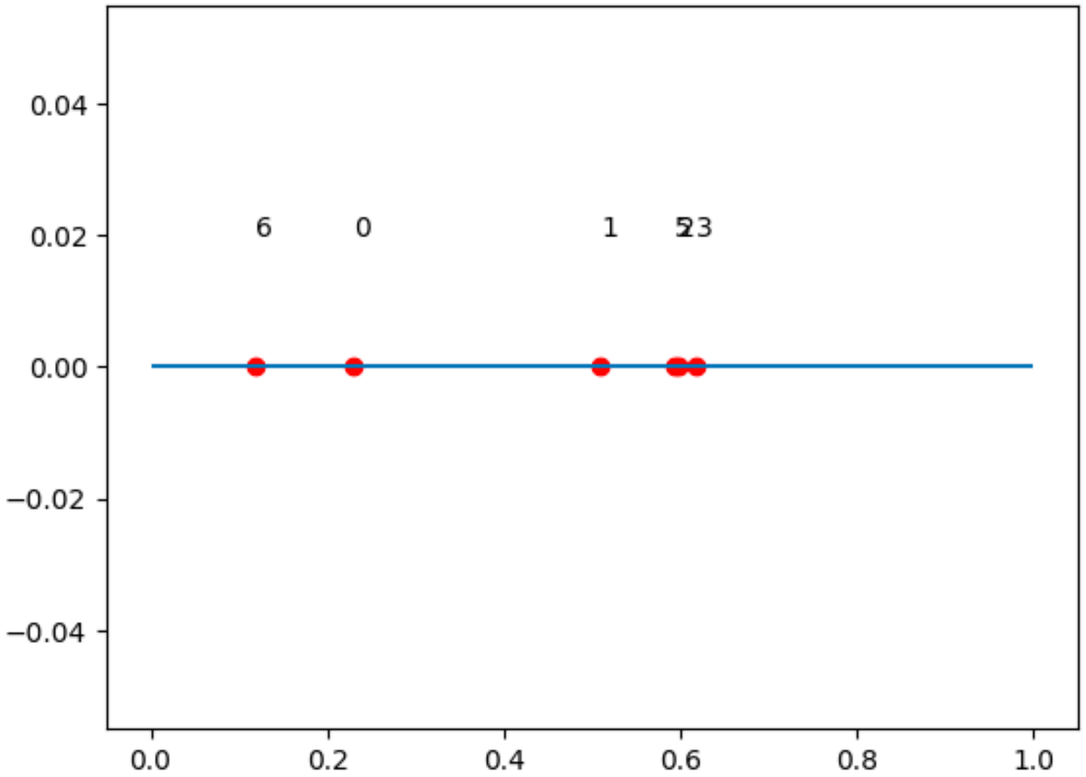




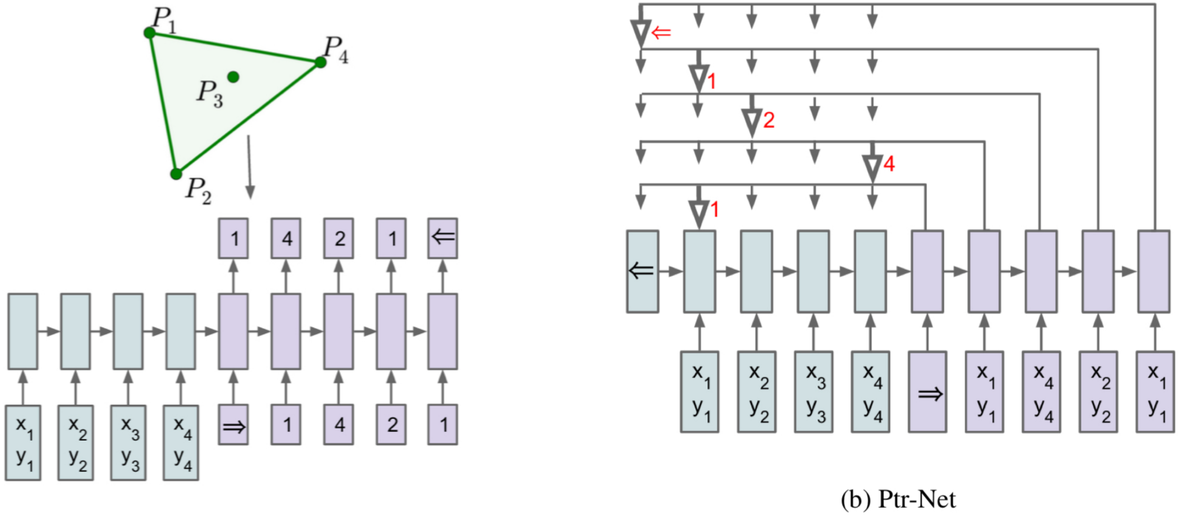
일반화…?????? wtf?

n=5로 sort 훈련시켰을 때 5이하 잘됨

-> 6부터 아예 안됨



Q? on Pointer Network



(b) Ptr-Net

-RNN = O(n)

-Attention = O(n^2)

이기 때문에 더 복잡한 알고리즘을 학습할 수 있다.
(computational capacity)

Q? on Pointer Network

-RNN = O(n)

-Attention = O(n^2)

이기 때문에 더 복잡한 알고리즘을 학습할 수 있다.
(computational capacity)

만약 이가 성립한다면 역으로 최적해의 계산복잡도를 추정할 수 있다!!!

Q? on Pointer Network

To generate exact data, we implemented the Held-Karp algorithm [18] which finds the optimal solution in $O(2^n n^2)$ (we used it up to $n = 20$). For larger n, producing exact solutions is extremely costly, therefore we also considered algorithms that produce approximated solutions: A1 [19] and A2 [20], which are both $O(n^2)$, and A3 [21] which implements the $O(n^3)$ Christofides algorithm. The latter algorithm is guaranteed to find a solution within a factor of 1.5 from the optimal length. Table 2 shows how they performed in our test sets.

n	OPTIMAL	A1	A2	A3	PTR-NET
5	2.12	2.18	2.12	2.12	2.12
10	2.87	3.07	2.87	2.87	2.88
50 (A1 TRAINED)	N/A	6.46	5.84	5.79	6.42
50 (A3 TRAINED)	N/A	6.46	5.84	5.79	6.09

Q? on Pointer Network

추후 <think>와 같이 sort 과정을 담아서

- bubble sort: O(n^2)
- quick sort: O(nlogn)…

computational capacity에 대한 고찰을 해볼 예정

Q? on Pointer Network

in fact: 알고리즘을 설계하는 뇌의 계산복잡도를 생각하면 이보다 작을 수 있다. (function calling의 시대… = this?)

~~뇌의 계산 복잡도가 O(n^2)라는 건 좀 슬프네~~

Q2? on Pointer Network

We note that any permutation of the sequence $\mathcal{C}^{\mathcal{P}}$ represents the same triangulation for $\mathcal{P}$, additionally each triangle representation C_i of three integers can also be permuted. Without loss of generality, and similarly to what we did for convex hulls at training time, we order the triangles C_i by their incenter coordinates (lexicographic order) and choose the increasing triangle representation[2]. Without ordering, the models learned were not as good, and finding a better ordering that the Ptr-Net could better exploit is part of future work.

(1, 2, 3) = (1, 3, 2) = (2, 1, 3) …

n!의 순열…중에 **무엇**을?

Set2Set: Orders Matters

Output의 관점: n!의 순열…중에 **무엇**을?

Input의 관점: 순서와 상관없이 **어떻게** 처리를..?

Set2Set: Orders Matters

Input의 관점: 순서와 상관없이 **어떻게** 처리를..?

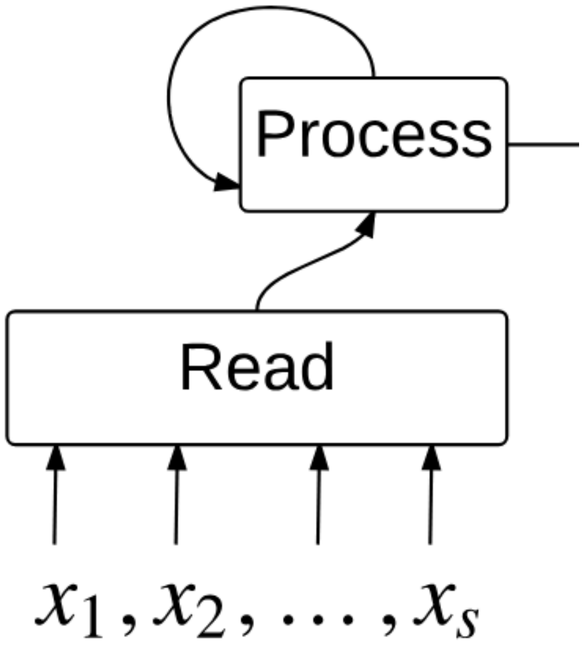
Embedding Table = **R**(vocab_size, embedding_dim)

i th vocab embedding = Table[**i**]

A simple approach which satisfies this, and which in fact has been commonly used for encoding sentences, is the bag-of-words approach. In this case, the representation is simply a reduction (e.g., addition) of counts, word embeddings, or similar embedding functions, and is naturally permutation invariant. For language and other domains which are naturally sequential, this is replaced with more complex encoders such as recurrent neural networks that take order into account and model higher order statistics of the data.

Set2Set: Orders Matters

Input의 관점: 순서와 상관없이 **어떻게** 처리를..?



추상적인 단계를 추가해보자!

-simple projection: xi -> mi

-read mi with attention (using query vector out on write)

Set2Set: Orders Matters

Input의 관점: 순서와 상관없이 **어떻게** 처리를..?

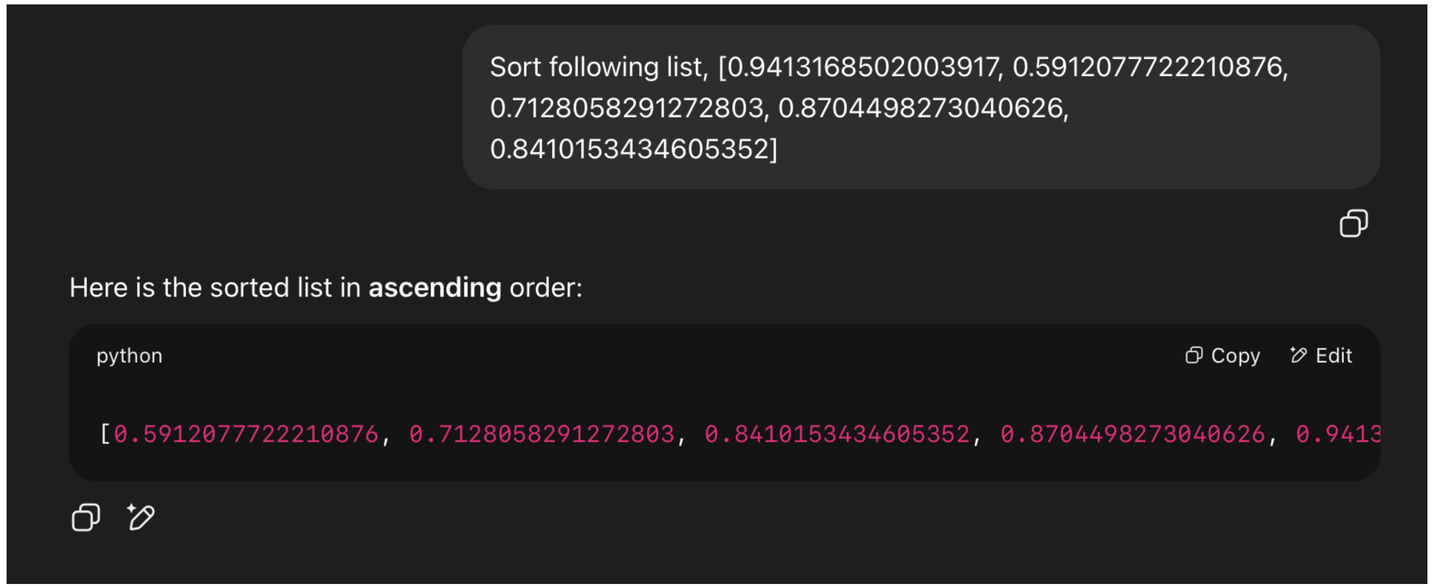
Table 1: The sorting experiment: out-of-sample sorting accuracy for various problem sizes and processing steps, with or without glimpses. All the reported accuracies are shown after reaching 10000 training iterations, at which point all models had converged but none overfitted. Higher is better.

~n_layer

Length N	Ptr-Net		$P = 0$ step		$P = 1$ step		$P = 5$ steps		$P = 10$ steps	
glimpses	0	1	0	1	0	1	0	1	0	1
$N = 5$	81%	90%	65%	84%	84%	92%	88%	94%	90%	94%
$N = 10$	8%	28%	7%	30%	14%	44%	17%	57%	19%	50%
$N = 15$	0%	4%	1%	2%	0%	5%	2%	4%	0%	10%

Set2Set: Orders Matters

Input의 관점: 순서와 상관없이 **어떻게** 처리를..?



Set2Set: Orders Matters

Output의 관점: n!의 순열…중에 **무엇**을?

- Language Modeling: “This is a sentence.” <-> “a is This <pad> . sentence”
- Parsing: BFS vs DFS
- Combinational Problem

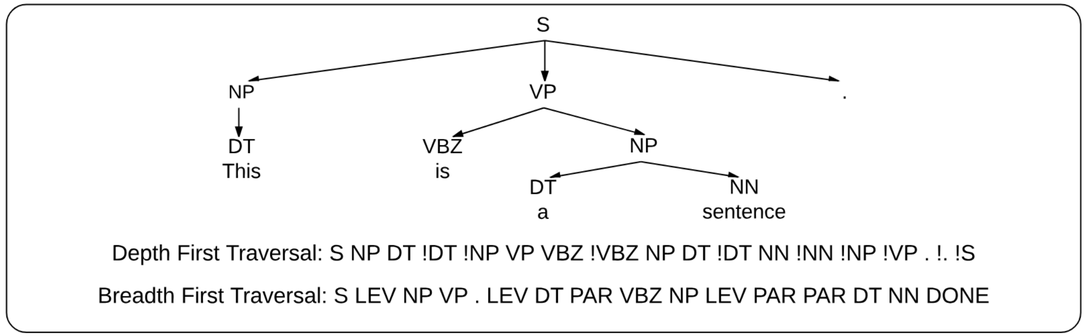


Figure 2: Depth first and breadth first linearizations of a parse tree which shows our different setups for output ordering in the parsing task.

Set2Set: Orders Matters

Output의 관점: n!의 순열…중에 **무엇**을?

$$P(y_1, y_2, \ldots, y_T) = \prod_{t=1}^{T} P(y_t | y_1, y_2, \ldots, y_{t-1})$$

we don’t know how they interact… ㅠㅠ

-> natural order of language gives nice coffee

Set2Set: Orders Matters

Output의 관점: n!의 순열…중에 **무엇**을?

-data large -> able to learn

-marginal distribution peaky(thus deterministic) -> able to learn

-in all other case -> learn **easy with optimal order**

$$\theta^\star = \arg\max_\theta \sum_i \max_{\pi(X_i)} \log p(Y_{\pi(X_i)}|X_i;\theta)$$

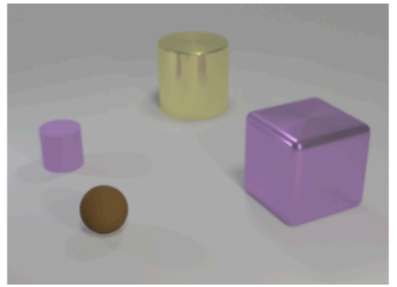
Relational Network

깊이 있는 질문이란 무엇일까?

- classification, detection? -> LUT(look up table)
- context dependent question=relation

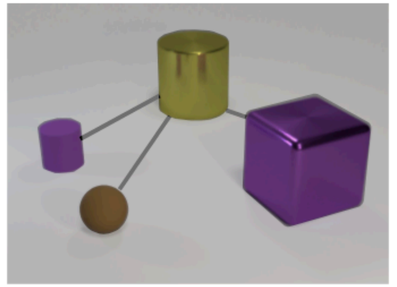
Non-relational question:

What is the size of the brown sphere?



Relational question:

Are there any rubber things that have the same size as the yellow metallic cylinder?



Relational Network

seemingly simple relational inferences are remarkably difficult for powerful neural network architectures such as convolutional neural networks (CNNs) and multi-layer perceptrons (MLPs).

새로운 Framework의 제안

relation

$$\mathbf{RN}(O) = f_\phi \left(\sum_{i,j} g_\theta(o_i, o_j) \right),$$

Prev. Relational Network

- Graph Neural Network
- Gated Graph Sequence Neural Network
- Interaction Network

Towards a simple update rule, cont'd

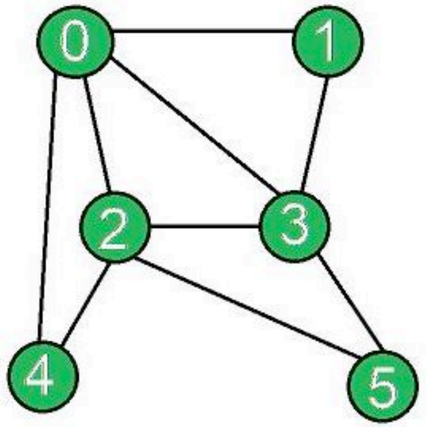
- Firstly, this update rule discards the central node.
  - Provide a simple correction: $\mathbf{H}' = \sigma(\tilde{\mathbf{A}}\mathbf{H}\mathbf{W})$

where $\tilde{\mathbf{A}} = \tilde{\mathbf{A}} + \mathbf{I}$

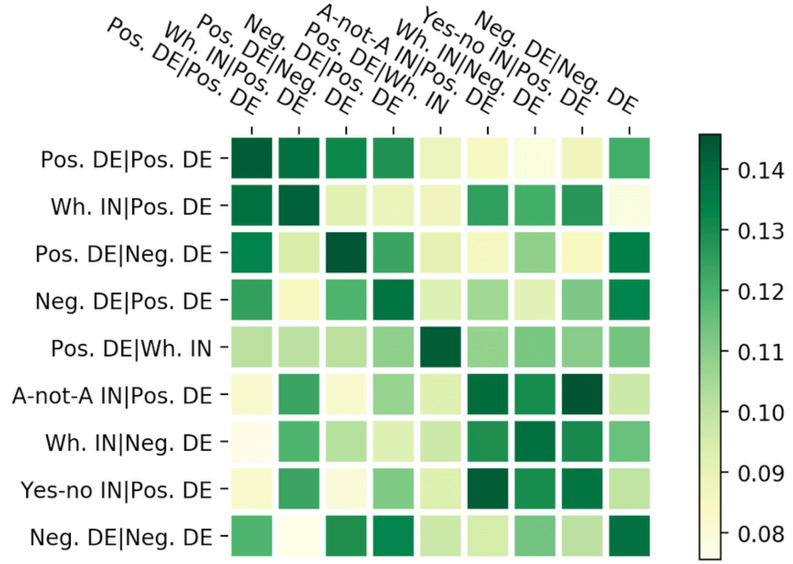
- The update rule can now be rewritten, node-wise, as:

$$\vec{h}_i' = \sigma\left(\sum_{j \epsilon N_i} \mathbf{W}\vec{h}_j\right)$$

Prev. Relational Network



	0	1	2	3	4	5
0	0	1	1	1	1	0
1	1	0	0	1	0	0
2	1	0	0	1	1	1
3	1	1	1	0	0	1
4	1	0	1	0	0	0
5	0	0	1	1	0	0



output -> sigma(AHW)

self-attention은 adjacency matrix를

내적 연산으로 만든다!!

Relational Network

- learn to infer relations: 모든 페어에 대한 정보를 고려 -> 관계의 학습
- data efficient: 하나의 g로 interaction의 고려
- operate on set of objects: invariant to order